Metasploit — Steps to Check Up

Cerberus pentest framework · VS Code terminal runbook (PowerShell)

verification guide · commit 8fd3935

Run these steps in the **VS Code integrated terminal** (PowerShell) to confirm the Metasploit integration is installed, configured, and working. Every step lists the command and the **expected output** so you can verify each one.

> **Prerequisite:** the backend virtual environment must exist. If missing, create it first:
>
> ```
> cd backend
> python -m venv .venv
> .venv\Scripts\python -m pip install -r requirements.txt -r requirements-dev.txt
> ```

0 Prerequisites — Metasploit present

```
msfconsole --version
msfrpcd --version
```

Expected: Metasploit Framework version banners. If these are unknown commands, install Metasploit first — the integration drives it via its RPC daemon.

1 Start the Metasploit RPC daemon

Open a **second** VS Code terminal (Terminal → New Terminal) and leave it running:

```
msfrpcd -P YourStrongPassword -S -a 127.0.0.1
```

Expected: daemon starts and listens on `127.0.0.1:55553`. (`-S` = plain HTTP locally; omit it and set `MSF_RPC_SSL=true` for TLS.)

2 Enable the integration in `.env`

Edit `backend/.env` (the file already contains the block — just change the values):

```
MSF_ENABLED=true
MSF_RPC_HOST=127.0.0.1
MSF_RPC_PORT=55553
MSF_RPC_PASSWORD=YourStrongPassword
MSF_RPC_SSL=false
MSF_RPC_TIMEOUT=10
```

> **Restart the backend** after editing `.env` — settings are read at startup.

3 Unit tests — the integration suite

15 tests, including a real in-process fake `msfrpcd` (stdlib XML-RPC server), so these pass **even without a live Metasploit**:

```
cd backend
.venv\Scripts\python -m pytest tests/unit/test_pentest_tools.py -v
```

Expected: `15 passed` — covers connect/auth, console execution, exploit refusal, RoE escalation, port parsing, and fail-soft.

4 Full backend test suite

```
.venv\Scripts\python -m pytest tests/unit -q
```

Expected: `76 passed` (61 framework tests + 15 tool-integration tests).

5 Live smoke — framework + tool layer

```
cd ..
backend\.venv\Scripts\python scripts\live_tests\pentest_framework_smoke.py
```

Expected output (tail):

```
tools registered: 1
  tool: metasploit (status=disconnected, enabled_by_default=False)
module: metasploit_scanner (tools category)
[INFO    ] Metasploit integration disabled      ← with MSF_ENABLED=false
SAFETY GATE OK — out-of-scope refused: ...       ← scope gate proven
```

With `MSF_ENABLED=true` (step 2) and the daemon up (step 1), the same smoke run instead reports a real scan finding: `[INFO] Metasploit auxiliary/scanner/portscan/tcp: N open ports on 127.0.0.1`.

6 Live check against the REAL daemon

Proves the adapter talks to your actual `msfrpcd` — paste into the terminal from `backend/`:

```
.venv\Scripts\python -c @"
import asyncio
from app.ai_engine.pentest import MetasploitAdapter

async def main():
    adapter = MetasploitAdapter()
    version = await adapter.connect()
    print("connected:", version)
    modules = await adapter.list_modules("auxiliary")
    print("auxiliary modules available:", len(modules))
    print("sample:", modules[:5])
    await adapter.disconnect()

asyncio.run(main())
"@
```

Expected: a version dict (e.g. `{'version': '6.x.x', ...}`) and a non-zero count of auxiliary modules from your real Metasploit install.

7 Run a real scan through the engine

The full production path — SafetyGuard → module → adapter → Metasploit console → parsed findings. Replace the target with an **in-scope** host you are authorized to test:

```
.venv\Scripts\python -c @"
import asyncio
from app.ai_engine.pentest import (
    ExecutionEngine, PentestContext, ScopeConfig,
)

async def main():
    engine = ExecutionEngine()
```

```
    ctx = PentestContext(
        engagement_id="CHECKUP",
        target="http://YOUR_IN_SCOPE_HOST/",
        scope=ScopeConfig(allowed_targets=["YOUR_IN_SCOPE_HOST"]),
        config={
            "msf_module": "auxiliary/scanner/portscan/tcp",
            "msf_threads": 10,
        },
    )
    findings = await engine.execute("metasploit_scanner", ctx)
    for f in findings:
        print(f"[{f.severity.value.upper()}] {f.title}")
        print(f"  {f.description}")
        for ev in f.evidence:
            print(f"  evidence ({ev.type}): {ev.data}")

asyncio.run(main())
"@
```

Expected: an INFO finding whose title counts the open ports, with one `[observation]` evidence entry per OPEN port (`{'host': ..., 'port': N, 'protocol': 'TCP', 'state': 'OPEN'}`) plus a `[log]` entry holding the console-output tail.

8 Negative checks — prove the safety gates

Check	How	Expected
Exploit modules refused	Repeat step 7 with `"msf_module": "exploit/multi/handler"`	ERROR finding *"Exploit module refused"* — exploits need FULL impact in the RoE
Fail-soft when daemon down	Stop the msfrpcd terminal (step 1), rerun step 7	INFO finding *"Metasploit unavailable"* — engagement never crashes
Out-of-scope refused	Rerun step 7 with a target *not* in `allowed_targets`	`OutOfScopeError` raised before any module code runs
Wrong password	Set `MSF_RPC_PASSWORD` wrong, rerun step 6	`ToolUnavailableError` (login failed) — surfaced as an INFO finding in step 7

Troubleshooting

Symptom	Cause / fix
INFO "Metasploit integration disabled"	`MSF_ENABLED=false` — set it true (step 2) and restart the backend.
INFO "Metasploit unavailable"	`msfrpcd` not running / wrong host or port / firewall — redo step 1.
ERROR "login failed"	`MSF_RPC_PASSWORD` ≠ the `-P` value from step 1.
Step 6: `Connection refused`	Daemon not up, or bound to another address — check the msfrpcd terminal output.
Step 7: empty port list	No OPEN lines for the target — read the `[log]` console-output evidence for the module's own error (wrong module name, missing options, host unreachable).
Step 3/4: `No module named 'app'`	You are not inside `backend/` — `cd backend` first.

Cerberus backend · pentest framework · Metasploit integration (commit 8fd3935). Companion documents in this folder: "metasploit added files.pdf" (what was added) and "metasploit how to use.pdf" (operator guide).